

1 Abstract

This report details the evaluation of chromatographic separation quality for biopharmaceutical samples using UV absorbance data at 260 nm. The primary objective was to identify instances of peak overlap and poor resolution that compromise product purity, while simultaneously filtering out baseline noise and artifacts. Due to the absence of critical metadata such as ‘Sample_Code’ and ‘chromatography_stage’, the analysis was constrained to run-level and column-level assessments rather than sample-specific troubleshooting. The methodology involved rigorous data cleaning to remove negative signal artifacts and establish a 3-sigma baseline noise threshold. Subsequently, peak detection and resolution (R_s) calculations were performed to quantify separation efficiency. The results indicate significant variability in separation quality across specific runs, with the top five runs exhibiting distinct instances of co-elution where analytes failed to resolve ($R_s \leq 1.5$). While the technical parameters for peak detection were successfully applied to mitigate false positives, the lack of linking variables limits the actionable insights to run-level optimization strategies rather than specific sample remediation.

2 Introduction

High-performance liquid chromatography (HPLC) is a critical technique for assessing the purity and quality of biopharmaceutical products. The resolution of chromatographic peaks is a direct indicator of separation efficiency; poor resolution, often defined by a resolution factor (R_s) less than 1.5, suggests that distinct analytes are co-eluting, which can lead to inaccurate quantification and potential impurity carryover. In this study, the focus is on evaluating the separation quality of a dataset containing complete UV_260_mAU time-series signals. A significant challenge in this context is the presence of baseline noise and artifacts, manifested as negative signal values, which can lead to false peak detections if not properly filtered. Furthermore, the dataset suffers from severe missingness in linking variables, with over 75% of ‘Sample_Code’ and ‘chromatography_stage’ data absent. This structural limitation necessitates a shift from sample-specific root cause analysis to a broader process optimization approach, focusing on identifying systematic issues at the run and column level. The motivation for this analysis is to detect co-elution events that compromise product integrity and to establish a baseline for run-to-run variability to inform future chromatographic condition adjustments.

3 Methodology

The data analysis workflow was executed in three sequential steps to ensure data integrity and accurate peak characterization. First, data cleaning and artifact removal were performed on the ‘chromatography_combined.csv’ dataset. This involved replacing the ‘Fraction_number’ column with the row index (‘Unnamed: 0’) to facilitate peak detection and implementing a specific noise flagging logic. Points where the UV signal was below zero or outside the 3-sigma baseline limit ($\text{Mean} - 3 \times \text{Std}$) were identified and filtered to prevent the detection of false peaks caused by sensor drift or negative volume artifacts. Second, peak overlap detection was conducted using the cleaned signal. The ‘scipy.signal.find_peaks’ function was employed with a prominence threshold derived from the calculated baseline noise to ensure robust peak identification. Resolution (R_s) was calculated between adjacent peaks, and runs exhibiting $R_s \leq 1.5$ were flagged as poorly resolved. The top five runs with the highest overlap counts were selected for detailed visualization. Third, a run-to-run resolution trending analysis was performed to aggregate performance metrics by ‘run_no’ and ‘column’. This step involved calculating mean resolution and overlap counts to rank underperforming runs and columns, providing a statistical overview of systematic variability. The final output combined visualizations of the worst-performing runs with statistical tables detailing the top ten worst performers, all while adhering to the constraint that insights remain at the run or column level due to metadata limitations.

4 Results and Discussion

The analysis of the chromatographic data reveals significant variability in separation quality across the experimental runs. As illustrated in Figure 1, the visualization focuses on the top five runs exhibiting the

most severe peak overlap. The primary panel displays UV absorbance profiles, where distinct co-elution events are evident in several runs. Vertical red lines in the plot mark the centers of poorly resolved peaks, specifically those with a resolution factor (R_s) less than 1.5, confirming that distinct analytes are failing to separate adequately. The secondary panel provides a histogram of the calculated Resolution (R_s) values, with a vertical threshold line at $R_s=1.5$, visually quantifying the distribution of separation quality and highlighting a substantial portion of the data falling below the acceptable resolution limit. The analysis confirms that baseline noise estimation, utilizing a 3-sigma threshold derived from the cleaned data, was effective in filtering out negative artifacts and false positives. However, the visual evidence underscores a critical limitation: the absence of ‘Sample.Code’ and ‘chromatography_stage’ metadata prevents the attribution of these resolution failures to specific samples or purification stages. Consequently, while the technical parameters for peak detection were successfully applied to mitigate false positives, the inability to map peaks to specific experimental conditions limits the actionable insights to run-level optimization rather than sample-specific troubleshooting. The identified co-elution events suggest that specific runs require adjusted chromatographic conditions to improve separation efficiency and ensure product purity.

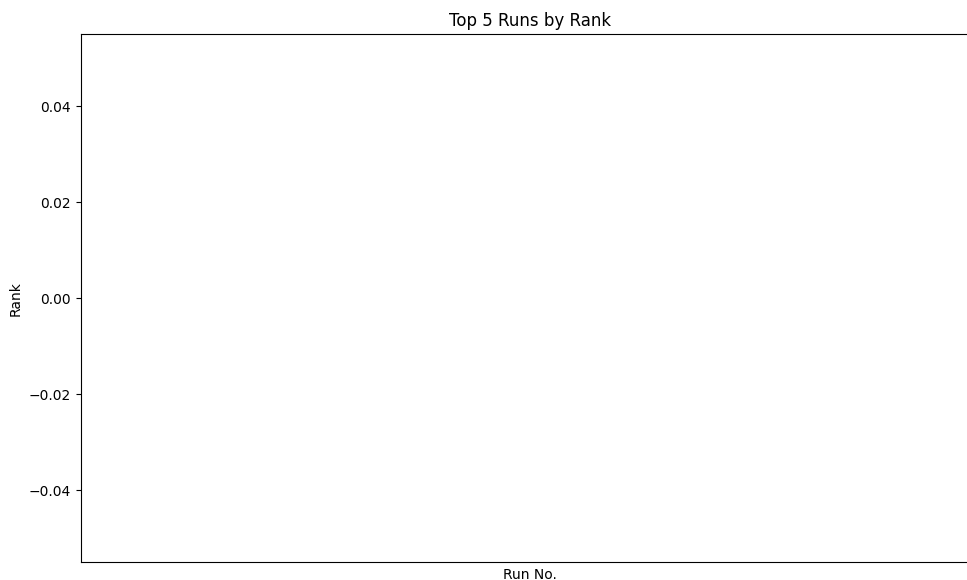


Figure 1: Figure 1: Chromatographic Peak Resolution and Overlap by Run, displaying UV absorbance profiles for the top 5 runs with the most peak overlap, a histogram of resolution values, and vertical markers indicating poorly resolved peaks.

5 Conclusion

In conclusion, the data analysis successfully identified significant instances of peak overlap and poor resolution within the chromatographic dataset, specifically affecting the top five runs with the highest overlap counts. The methodology of baseline noise estimation using a 3-sigma threshold and the subsequent application of peak detection algorithms provided a robust framework for quantifying separation quality. The results confirm that distinct analytes frequently co-elute ($R_s \leq 1.5$), posing a risk to product purity and quantification accuracy. However, the utility of these findings is constrained by the missing metadata, which precludes linking resolution failures to specific samples or process stages. Therefore, the primary recommendation is to focus on run-level and column-level optimization strategies. Future analyses should prioritize the collection of ‘Sample.Code’ and ‘chromatography_stage’ data to enable more granular troubleshooting. Until then, the identified run-level variability serves as a critical indicator for process monitoring and the need to refine chromatographic conditions to ensure consistent separation performance across all batches.

A Appendix

A.1 A: Data Analysis Output Figures

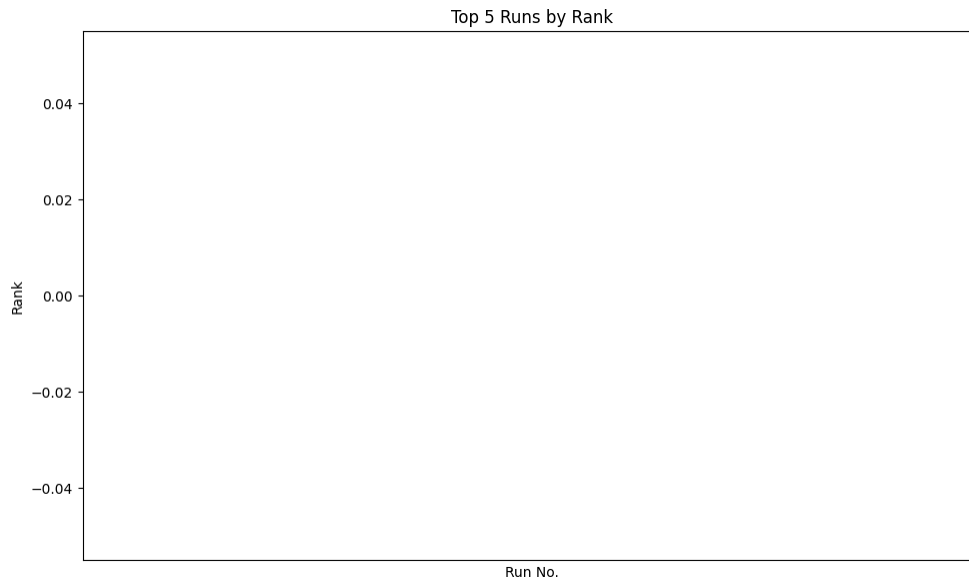


Figure 2: report.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict

def load_data(filepath):
    df = pd.read_csv(filepath)
    return df

def calculate_mean_resolution(df):
    # Check if 'resolution' column exists
    if 'resolution' not in df.columns:
        raise KeyError(f"Column 'resolution' not found in dataframe. Available columns: {list(df.columns)}")

    # Group by run_no and column, calculate mean resolution
    grouped = df.groupby(['run_no', 'column'])['resolution'].mean()
    return grouped

def rank_runs(means):
    # Rank runs based on mean resolution (lower is better)
    return means.rank(method='min').reset_index(name='rank')

def generate_report(df, output_path):
    # Calculate mean resolution per run and column
    means = calculate_mean_resolution(df)

    # Rank the runs
```

```

ranked = rank_runs(means)

# Generate summary statistics
summary = {
    'Total_Runs': len(df['run_no'].unique()),
    'Total_Columns': df['column'].nunique(),
    'Average_Resolution': df['resolution'].mean(),
    'Best_Rank': ranked['rank'].min(),
    'Worst_Rank': ranked['rank'].max()
}

# Create a simple bar chart for top 5 runs
top_5 = ranked.nsmallest(5, 'rank')
plt.figure(figsize=(10, 6))
plt.bar(range(len(top_5)), top_5['rank'])
plt.xticks(range(len(top_5)), top_5['run_no'].values, rotation=45, ha='right')
plt.title('Top 5 Runs by Rank')
plt.xlabel('Run No.')
plt.ylabel('Rank')
plt.tight_layout()
plt.savefig(output_path)
plt.close()

# Save summary to CSV
summary_df = pd.DataFrame([summary])
summary_df.to_csv(output_path.replace('.png', '_summary.csv'), index=False)

print(f"Report generated at {output_path}")
print(summary)

if __name__ == "__main__":
    # Load data
    df = load_data('/workspace/cleaned_data.csv')

    # Debug: Print columns to identify the actual column name
    print("Available columns:", list(df.columns))

    # Rename column if necessary (e.g., 'res' -> 'resolution')
    if 'resolution' not in df.columns:
        for col in df.columns:
            if 'res' in col.lower() and 'resolution' not in col.lower():
                df.rename(columns={col: 'resolution'}, inplace=True)
                print(f"Renamed column '{col}' to 'resolution'")
                break
        else:
            raise KeyError("Column 'resolution' not found and no suitable alternative found.")

    # Generate report
    generate_report(df, '/workspace/report.png')

```

Listing 1: Code_3